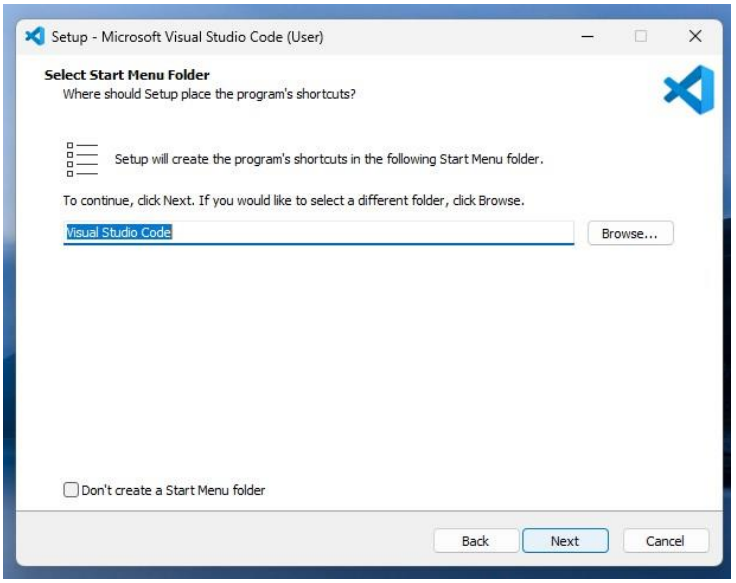
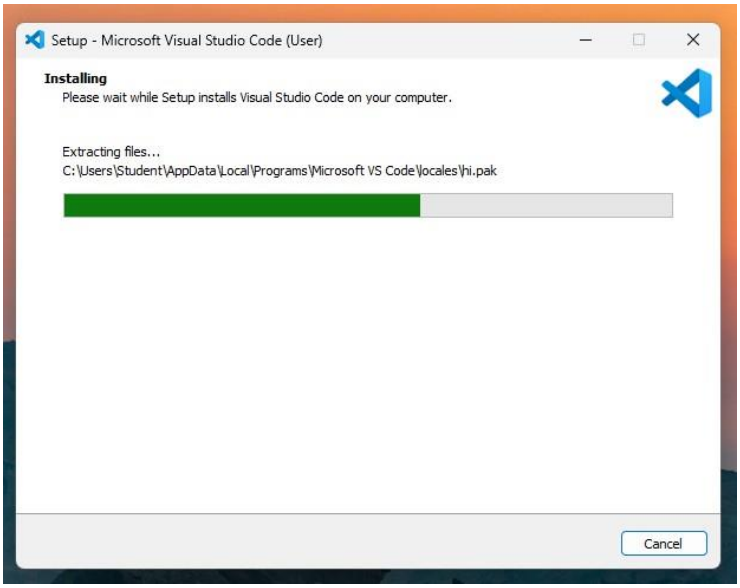
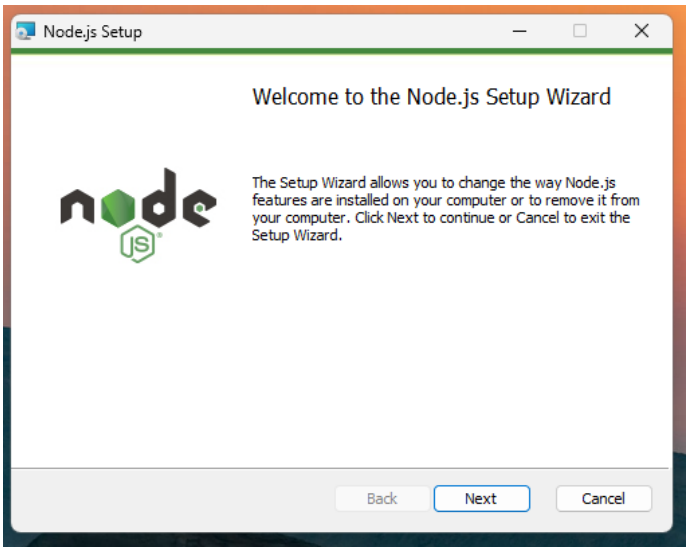
Practical 1

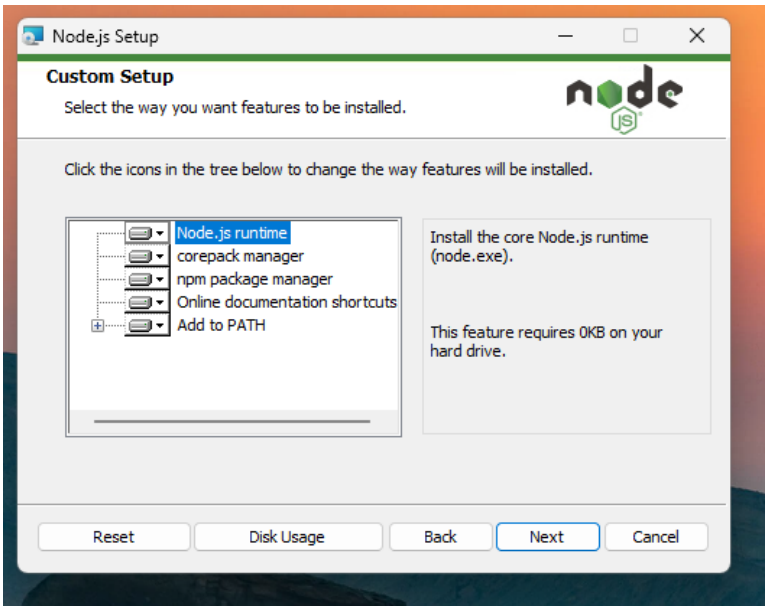
a. Install Node.js and Visual Studio Code, and verify installation.

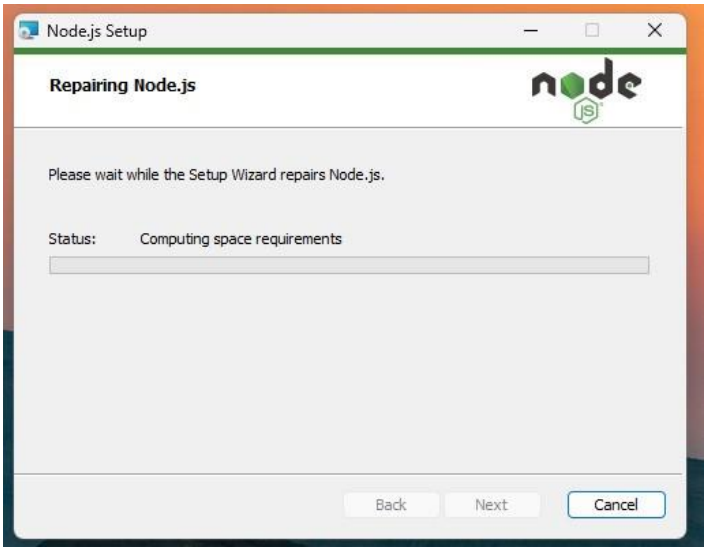
Output:

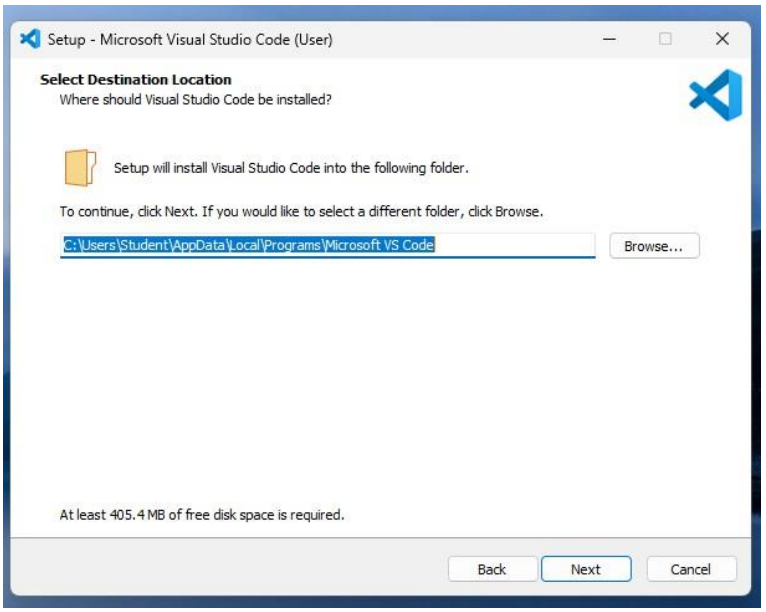


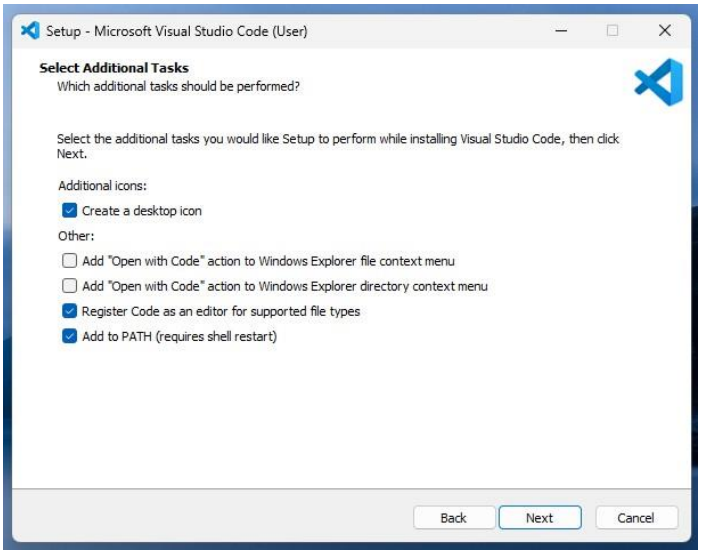


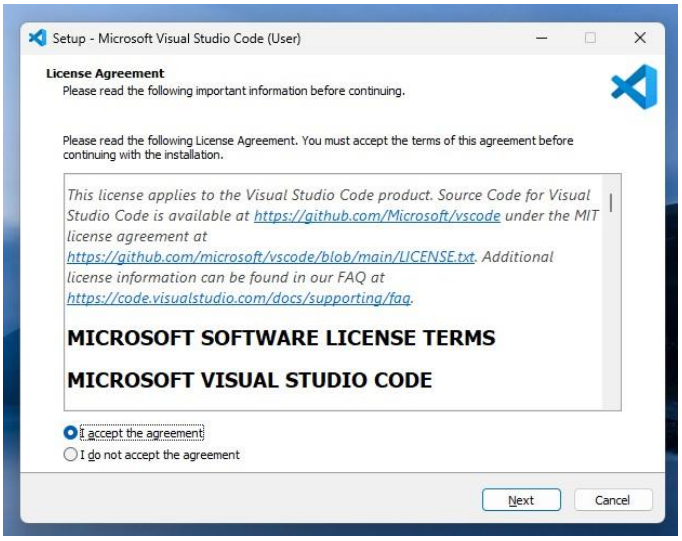

Node.js Setup
Welcome to the Node.js Setup Wizard
The Setup Wizard allows you to change the way Node.js features are installed on your computer or to remove it from your computer. Click Next to continue or Cancel to exit the Setup Wizard.
Back
Next
Cancel


Node.js Setup
Custom Setup
Select the way you want features to be installed.
Click the icons in the tree below to change the way features will be installed.
Node.js runtime
corepack manager
npm package manager
Online documentation shortcuts
Add to PATH
Install the core Node.js runtime (node.exe).
This feature requires 0KB on your hard drive.
Reset
Disk Usage
Back
Next
Cancel


Node.js Setup
Repairing Node.js
Please wait while the Setup Wizard repairs Node.js.
Status: Computing space requirements
Back
Next
Cancel


Setup - Microsoft Visual Studio Code (User)
Select Destination Location
Where should Visual Studio Code be installed?
Setup will install Visual Studio Code into the following folder.
To continue, click Next. If you would like to select a different folder, click Browse.
C:\Users\Student\AppData\Local\Programs\Microsoft VS Code
Browse...
At least 405.4 MB of free disk space is required.
Back
Next
Cancel


Setup - Microsoft Visual Studio Code (User)
Select Additional Tasks
Which additional tasks should be performed?
Select the additional tasks you would like Setup to perform while installing Visual Studio Code, then click Next.
Additional icons:
Create a desktop icon
Other:
Add "Open with Code" action to Windows Explorer file context menu
Add "Open with Code" action to Windows Explorer directory context menu
Register Code as an editor for supported file types
Add to PATH (requires shell restart)
Back
Next
Cancel


Setup - Microsoft Visual Studio Code (User)
License Agreement
Please read the following important information before continuing.
Please read the following License Agreement. You must accept the terms of this agreement before continuing with the installation.
This license applies to the Visual Studio Code product. Source Code for Visual Studio Code is available at https://github.com/Microsoft/vscode under the MIT license agreement at https://github.com/microsoft/vscode/blob/main/LICENSE.txt. Additional license information can be found in our FAQ at https://code.visualstudio.com/docs/supporting/faq.
MICROSOFT SOFTWARE LICENSE TERMS
MICROSOFT VISUAL STUDIO CODE
I accept the agreement
I do not accept the agreement
Next
Cancel

b. Create a simple Node.js file and run it using the terminal.

Code:

```
console. 1og("He11o");
```

Output:



c. Write a Node.js script demonstrating basic JavaScript commands ((console log, variables, and arithmetic operations).

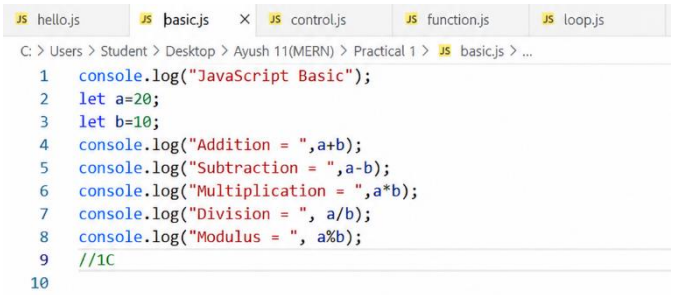
Code:

```
// Console Output
console.log("JavaScript Basics");

// Variables
let a = 20;
let b = 10;

// Arithmetic Operations
console.log("Addition =", a + b);
console.log("Subtraction =", a - b);
console.log("Multiplication =", a * b);
console.log("Division =", a / b);
console.log("Modulus =", a % b);
```

Output:





d. Create and run a simple program using variables and functions.

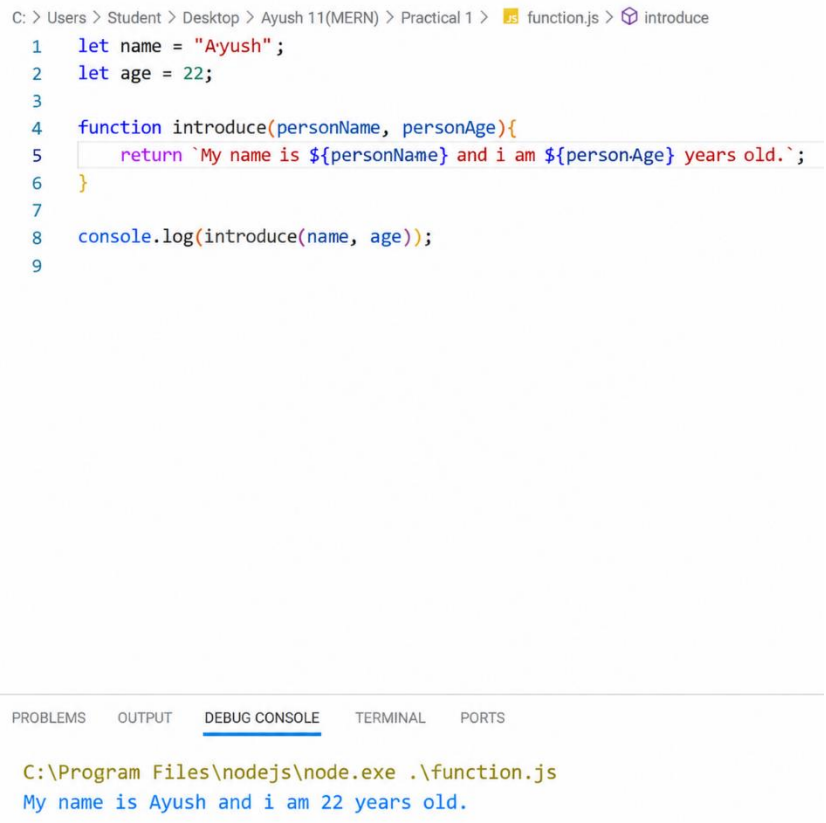
Code:

```
// Variables
let name = "Ayush";
let age = 22;

// Function
function introduce(personName, personAge) {
   return `My name is ${personName} and I am ${personAge} years old.`;
}

// Function Call
console.log(introduce(name, age));
```

Output:



e. Demonstrate use of operators and control statements in JavaScript

Code:

```
let num1 = 20;
let num2 = 30;

console.log("a>b", num1 > num2);
console.log("a==b", num1 == num2);

if (num1 > num2) {
   console.log("num1 is greater than num2");
} else {
   console.log("num2 is greater than num1");
}
```

Output:



f. Write programs using loops and template literals.

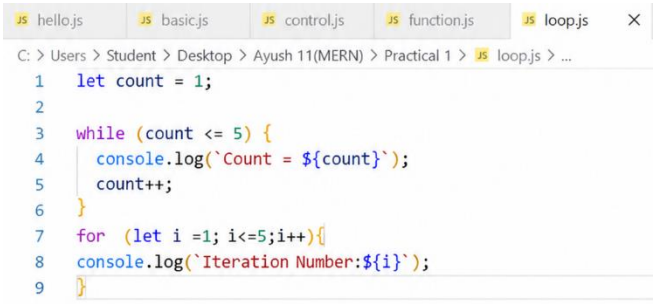
Code:

```
let count = 1;

while(count <= 5) {
   console.log(`Count = ${count}`);
   count++;
}

for(let i = 1; i <= 5; i++) {
   console.log(`Iteration Number: ${i}`);
}
```

Output:





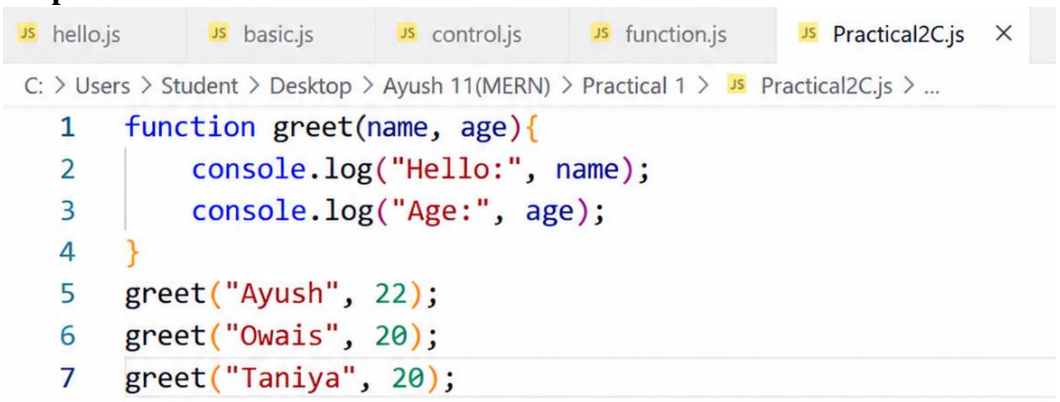
Practical 2

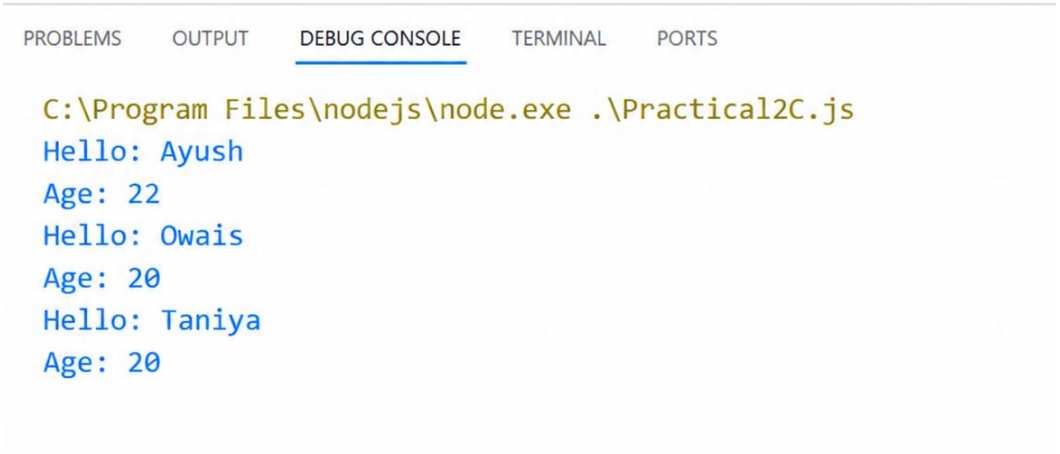
a. Create functions and call them with parameters.

Code:

```
function greet(name, age){
   console.log("Hello:", name);
   console.log("Age:", age);
}
greet("Ayush", 22);
greet("Owais", 20);
greet("Taniya", 20);
```

Output:



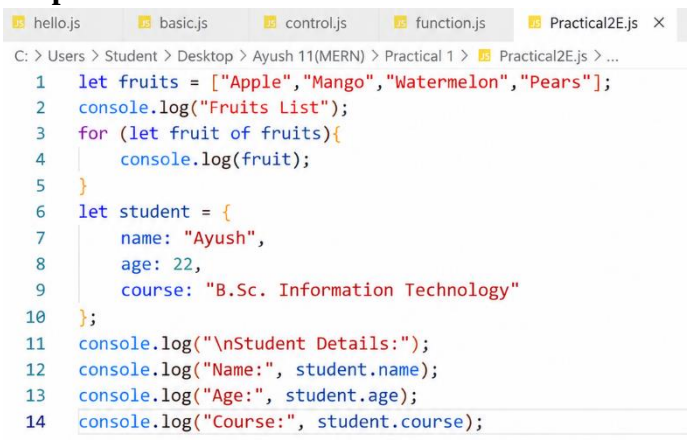


b. Work with arrays and objects in JavaScript.

Code:

```
let fruits = ["Apple","Mango","Watermelon","Pears"];
console.log("Fruits List");
for (let fruit of fruits){
   console.log(fruit);
}
let student = {
   name: "Ayush",
   age: 22,
   course: "B.Sc. Information Technology"
};
console.log("\nStudent Details:");
console.log("Name:", student.name);
console.log("Age:", student.age);
console.log("Course:", student.course);
```

Output:



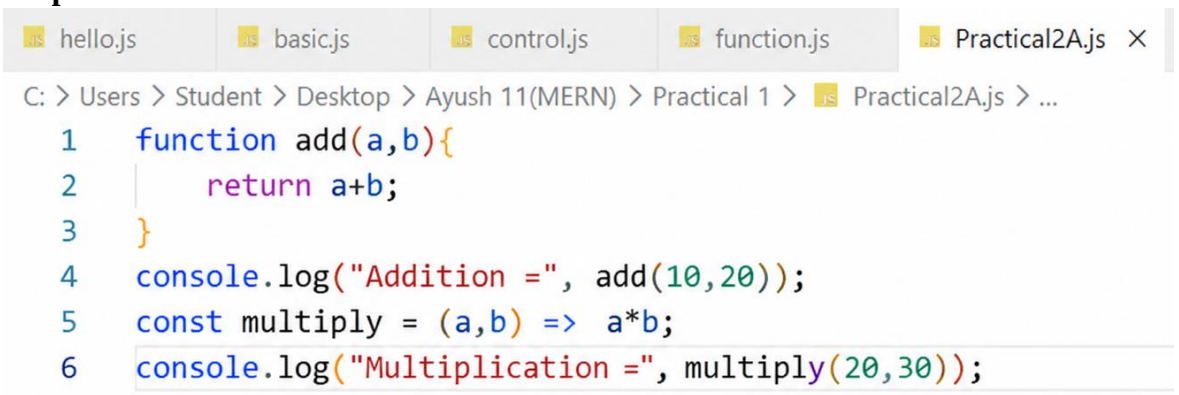


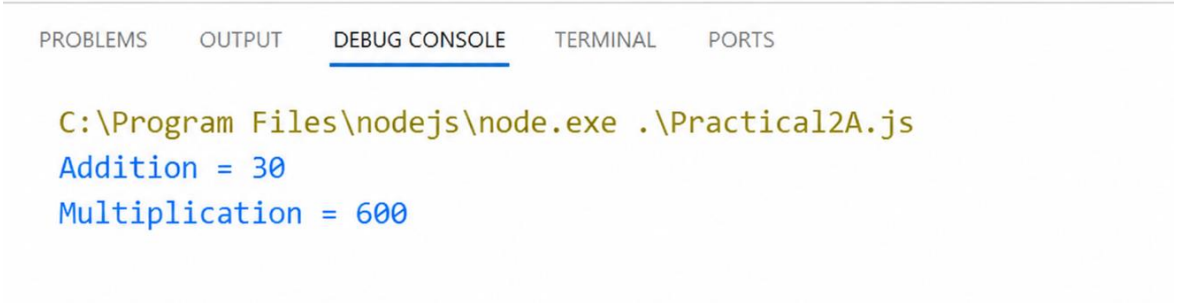
c. Demonstrate use of arrow functions.

Code:

```
function add(a,b){
   return a+b;
}
console.log("Addition =", add(10,20));
const multiply = (a,b) => a*b;
console.log("Multiplication =", multiply(20,30));
```

Output:





d. Write a program using array methods (map, filter, reduce).

Code:

```
let numbers = [10,20,30,40,50];
let doubled= numbers.map(num=> num*2);
console.log("Doubled Array:");
console.log(doubled);
let greaterThan25 = numbers.filter(num=> num>25);
console.log("\nNumbers Greater than 25:");
console.log(greaterThan25);
let sum = numbers.reduce((total, num)=> total+num, 0);
console.log("\nSum of Elements");
console.log(sum);
```

Output:



e. Create a program that manipulates objects and displays output.

Code:

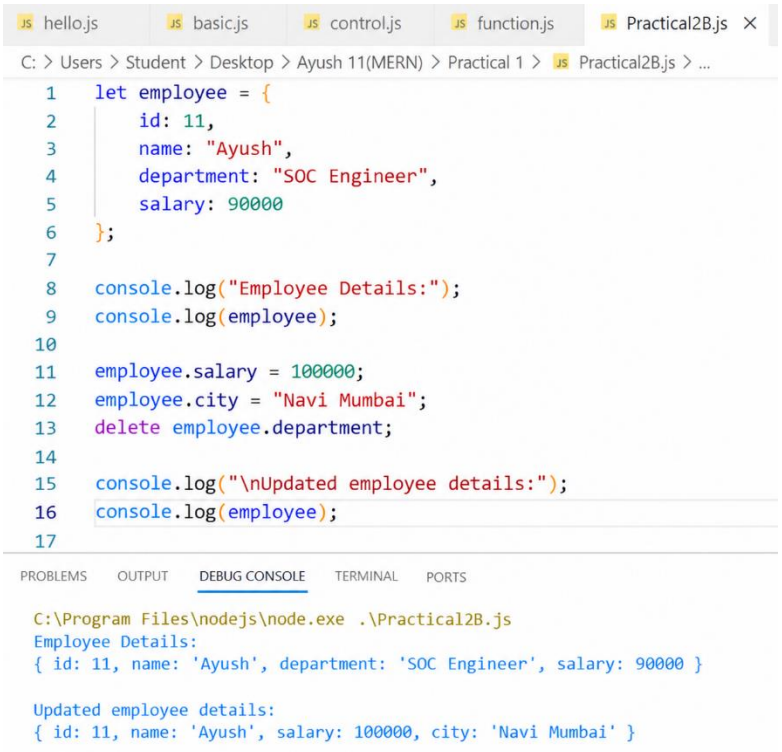
```
let employee = {
   id: 11,
   name: "Ayush",
   department: "SOC Engineer",
   salary: 90000
};

console.log("Employee Details:");
console.log(employee);


employee.salary = 100000;
employee.city = "Navi Mumbai";
delete employee.department;

console.log("\nUpdated employee details:");
console.log(employee);
```

Output:



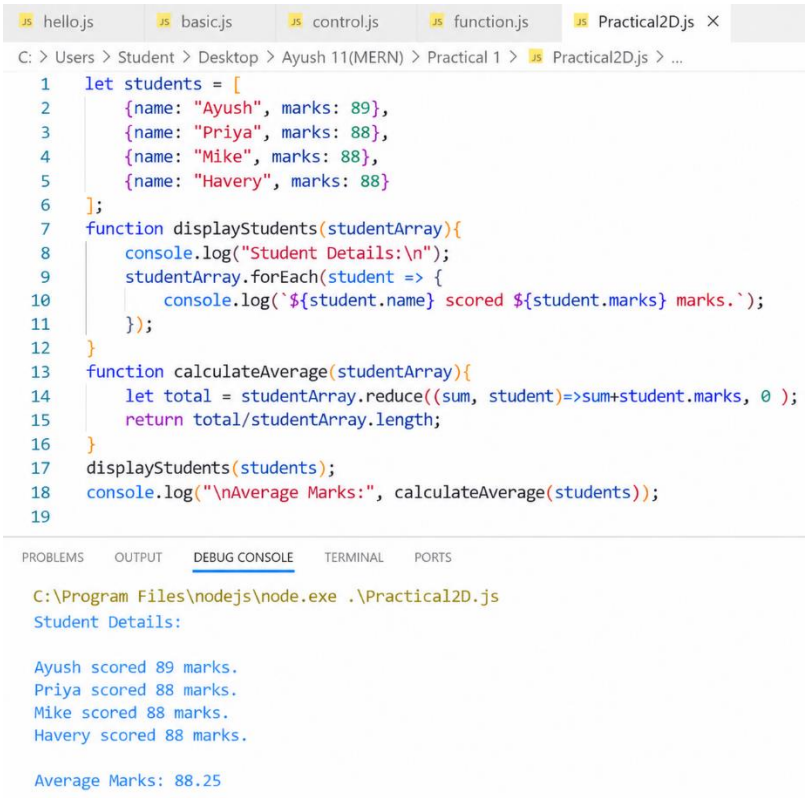
\

f. Implement a small JavaScript program combining functions, arrays, and objects.

Code:

```
let students = [
   {name: "Ayush", marks: 89},
   {name: "Priya", marks: 88 },
   {name: "Mike", marks: 88},
   {name: "Havery", marks: 88}
];
function displayStudents(studentArray){
   console.log("Student Details:\n");
   studentArray.forEach(student => {
      console.log(`${student.name} scored ${student.marks} marks.`);
   });
}
function calculateAverage(studentArray){
   let total = studentArray.reduce((sum, student)=>sum+student.marks, 0 );
   return total/studentArray.length;
}
displayStudents(students);
console.log("\nAverage Marks:", calculateAverage(students));
```

Output:



Practical 3

Aim: React Introduction and JSX

a. Create a React application using Vite/CRA and display "Welcome to MERN Stack development".

b. Display dynamic data using JSX.

c. Declare variables and write a function to calculate the sum of two numbers and display the result.

d. Apply basic CSS styling to a React component

e. Create a simple functional component and render it in App.js.

f. Create a React component that displays the current date and time dynamically using JavaScript.

Greeting.jsx

```
function Greeting() {
  return (
    <div>
      <h2>Hello from Greeting Component!</h2>
    </div>
  );
}

export default Greeting;
```

Date and Time Component (Requirement f)

DateTime.jsx

```
function DateTime() {
  const currentDate = new Date();

  return (
    <div>
      <h3>Current Date and Time</h3>
      <p>{currentDate.toLocaleString()}</p>
    </div>
  );
}

export default DateTime;
```

App Component (Requirements a, b, c, d)

App.jsx

```
import "./App.css";
import Greeting from "./Greeting";
import DateTime from "./DateTime";

function App() {
  // Dynamic Data using JSX
  const name = "Student";

  // Variables for Sum
  const num1 = 15;
  const num2 = 25;

  // Function to calculate sum
  function calculateSum(a, b) {
  return a + b;
  }

  return (
    <div className="container">
      {/* a. Display Message */}
      <h1>Welcome to MERN Stack Development</h1>

      {/* b. Dynamic Data */}
      <h2>Hello, {name}!</h2>

      {/* c. Sum of Two Numbers */}
      <p>
        Sum of {num1} and {num2} = {calculateSum(num1, num2)}
      </p>

      {/* e. Functional Component */}
      <Greeting />

      {/* f. Date and Time Component */}
      <DateTime />
    </div>
  );
}

export default App;
```

CSS Styling (Requirement d)

App.css

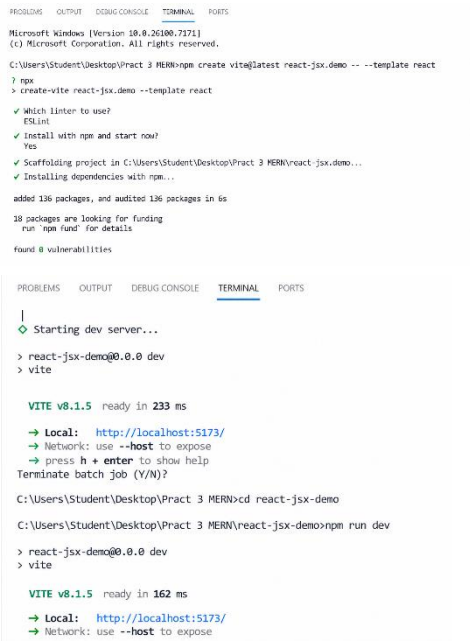
```
.container {
  text-align: center;
  margin-top: 50px;
  font-family: Arial, sans-serif;
}
```

```
h1 {
  color: darkblue;
}

h2 {
  color: green;
}

p {
  font-size: 18px;
  color: #333;
  background-color: #f2f2f2;
  padding: 10px;
  border-radius: 5px;
}
```

Output:





Practical 4

a. Create multiple components and render them on a single page.
b. Pass data between components using props
c. Use the useState hook to manage and update data.
d. Create a form component to accept user input and display it dynamically.
e. Display a list of items stored in the state using map().
f. Build a small interactive component (e.g., counter or to-do list).

Code:

```
//App.jsx
import React, { useState } from "react";
import "./App.css";

// Component 1: Header
function Header(props) {
  return <h1>{props.title}</h1>;
}

// Component 2: User Profile (using props)
function UserProfile({ name, age }) {
return (
    <div>
      <h2>User Profile</h2>
      <p>Name: {name}</p>
      <p>Age: {age}</p>
    </div>
  );
}

// Component 3: Form Component
function UserForm() {
  const [input, setInput] = useState("");
  const [displayName, setDisplayName] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    setDisplayName(input);
    setInput("");
  }

  return (
    <div>
      <h2>User Input Form</h2>
```

```
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          value={input}
          onChange={(e) => setInput(e.target.value)}
          placeholder="Enter your name"
        />

        <button type="submit">Submit</button>
      </form>

      <p>
        Entered Name: <b>{displayName}</b>
      </p>
    </div>
  );
}

// Component 4: Counter Component
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Counter</h2>
      <p>Count: {count}</p>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>

      <button onClick={() => setCount(count - 1)}>
        Decrease
      </button>
    </div>
  );
}

// Component 5: Todo List Component
function TodoList() {
  const [items, setItems] = useState([
    "Learn React",
    "Practice Components",
    "Build Projects"
  ]);

  const [task, setTask] = useState("");

  function addTask() {
    if (task !== "") {
```

```
      setItems([...items, task]);
      setTask("");
    }
  }

  return (
    <div>
      <h2>Todo List</h2>

      <input
        value={task}
        onChange={(e) => setTask(e.target.value)}
        placeholder="Add task"
      />

      <button onClick={addTask}>
        Add
      </button>

      <ul>
        {items.map((item, index) => (
          <li key={index}>
            {item}
          </li>
        ))}
      </ul>
    </div>
  );
}


// Main Component
function App() {
return (
    <div className="App">

      <Header title="React Components Demo" />

      <UserProfile
        name="Ayush"
        age="22"
      />

      <UserForm />

      <Counter />

      <TodoList />

    </div>
```

```
  );
}

export default App;
```

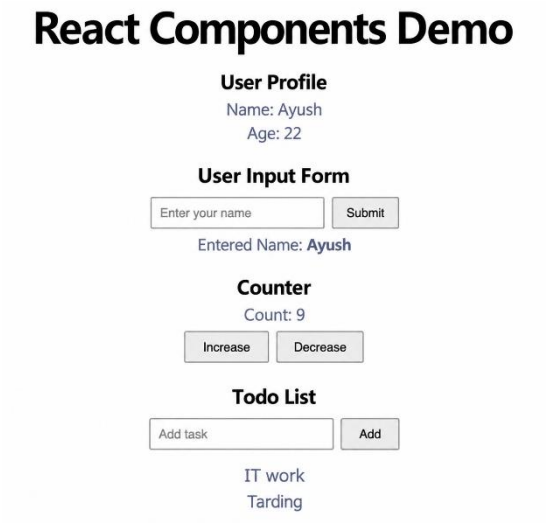
//App.css

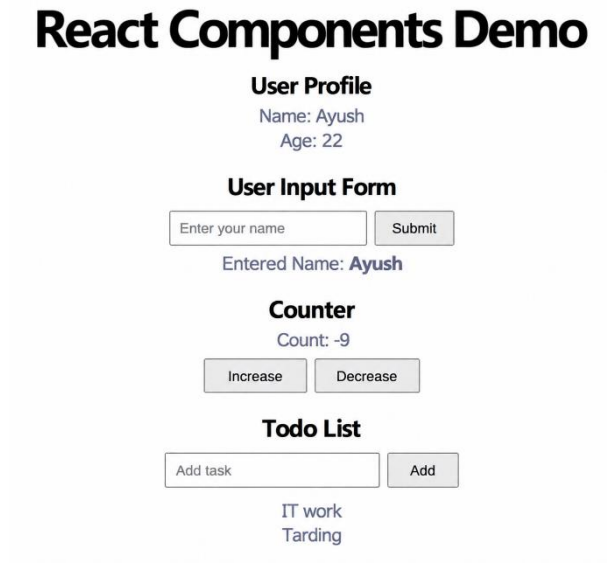
```
.App{
  text-align: center;
  font-family: Arial, sans-serif;
}
div{
  margin: 20px;
}

input{
  padding: 8px;
  margin: 5px;
}
button{
  padding: 8px 15px;
  margin: 5px;
  cursor: pointer;
}

li{
  list-style: none;
  margin: 5px;
}
```

Output:


React Components Demo
User Profile
Name: Ayush
Age: 22
User Input Form
Enter your name
Submit
Entered Name: Ayush
Counter
Count: 9
Increase
Decrease
Todo List
Add task
Add
IT work
Tarding


React Components Demo
User Profile
Name: Ayush
Age: 22
User Input Form
Enter your name
Submit
Entered Name: Ayush
Counter
Count: -9
Increase
Decrease
Todo List
Add task
Add
IT work
Tarding


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
Microsoft Windows [Version 10.0.26100.7171]
(c) Microsoft Corporation. All rights reserved.
C:\Users\Student\Desktop\Practical4MERN>cd react-jsx-demo
C:\Users\Student\Desktop\Practical4MERN\react-jsx-demo>npm run dev
> react-jsx-demo@0.0.0 dev
> vite
VITE v8.1.5 ready in 154 ms

Practical 5

React Routing and Data Handling

a. Install and configure React Router in the React application.

b. Create multiple pages and implement navigation between them.

c. Use Link and Routes to move between pages.

d. Fetch sample data from a public API using Axios.

e. Display fetched data in table or list format.

f. Implement search or filter functionality on displayed data.

Code:

App.jsx

```
import { Routes, Route, Link } from "react-router-dom";

import Home from "./pages/Home";
import About from "./pages/About";
import Users from "./pages/Users";

function App() {
  return (
    <div>
      <h1>React Routing Example</h1>

      <nav>
        <Link to="/">Home</Link> |{" "}
        <Link to="/about">About</Link> |{" "}
        <Link to="/users">Users</Link>
      </nav>

      <hr />

      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/users" element={<Users />} />
      </Routes>
    </div>
  );
}

export default App;
```

main.jsx

```
import React from "react";
```

```
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

About.jsx

```
function About() {
return (
    <div>
      <h2>About Page</h2>
      <p>This is the About Page.</p>
    </div>
  );
}

export default About;
```

Home.jsx

```
function Home() {
return (
    <div>
      <h2>Home Page</h2>
      <p>Welcome to the Home Page.</p>
    </div>
  );
}

export default Home;
```

Users.jsx

```
import { useEffect, useState } from "react";
import axios from "axios";

function Users() {
  const [users, setUsers] = useState([]);
  const [search, setSearch] = useState("");

  useEffect(() => {
    axios
      .get("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        setUsers(response.data);
      })
      .catch((error) => {
```

```
      console.log("Error:", error);
    });
  }, []);

  const filteredUsers = users.filter((user) =>
    user.name.toLowerCase().includes(search.toLowerCase())
  );

  return (
    <div>
      <h2>Users List</h2>

      <input
        type="text"
        placeholder="Search by Name..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />

      <br />
      <br />

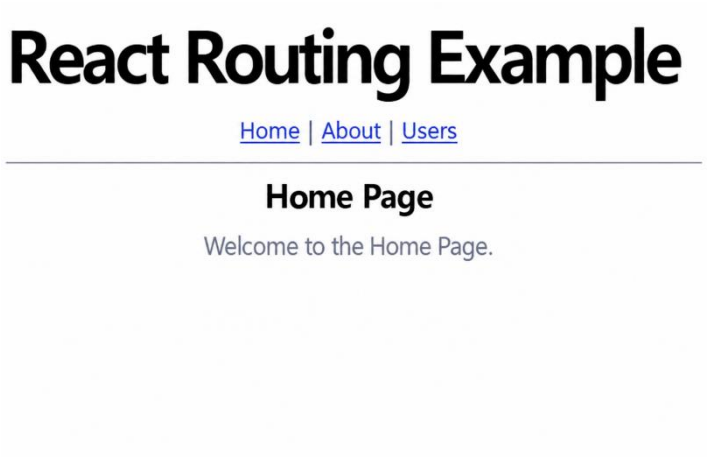
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Username</th>
            <th>Email</th>
            <th>Phone</th>
            <th>Website</th>
          </tr>
        </thead>

        <tbody>
          {filteredUsers.map((user) => (
            <tr key={user.id}>
              <td>{user.id}</td>
              <td>{user.name}</td>
              <td>{user.username}</td>
              <td>{user.email}</td>
              <td>{user.phone}</td>
              <td>{user.website}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

export default Users;

Output:







Practical 6

Node.js and Express Fundamentals

a. Install Express and create a simple server.

b. Display a message on the browser using the server response.

c. Create different routes in Express.

Code:

```
const express = require('express');
const app = express();
const PORT = 3000;
app.get('/',(req,res)=>{
   console.log("Home route was acessed");
   res.send('<h1>Welcome to my Express Server</h1>');
});
app.get('/about',(req,res)=>{
   console.log("About route was acessed");
   res.send('<h1>This is the about page</h1>');
});
app.get('/api/data',(req,res)=>{
   console.log("API Data route was acessed");
   res.json({
      name: 'Student',
      course:'Node.js and Express',
      status:'Learning Successful'
   });
});
app.get('/contact',(req,res)=>{
   console.log("Contact route was acessed");
   res.send('Contact us at: example@gmail.com')
});
app.listen(PORT,()=>{
   console.log(`Server is running successfully on http://localhost:${PORT}`);
});
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\Student\Desktop\Pr6>npm init -y
Wrote to C:\Users\Student\Desktop\Pr6\package.json:

{
  "name": "pr6",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
C:\Users\Student\Desktop\Pr6>npm install express

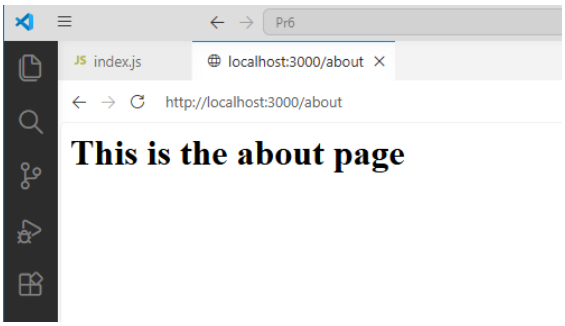
added 67 packages, and audited 68 packages in 2s

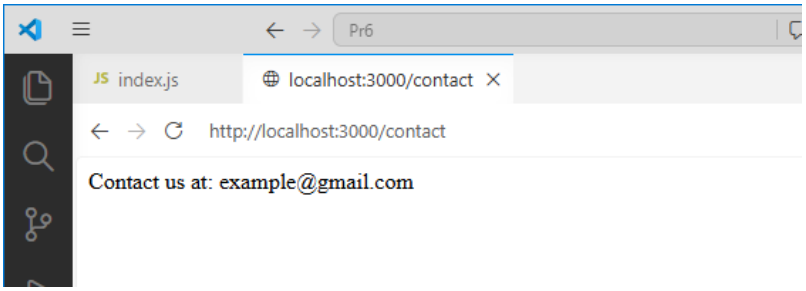
26 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

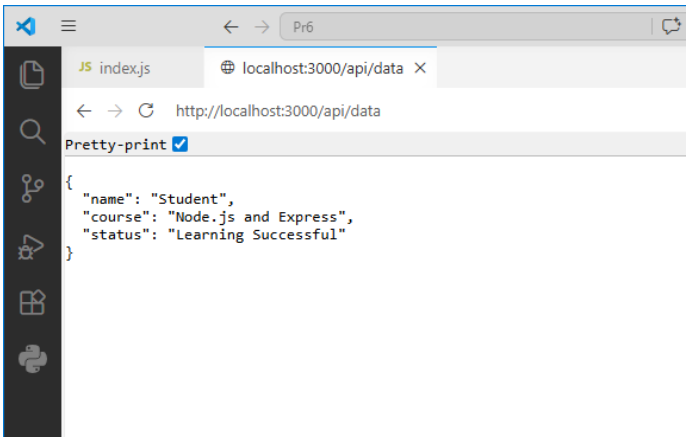
```
C:\Users\Student\Desktop\Pr6>node index.js
Server is runing successfully on http://localhost:3000
Home route was acessed
About route was acessed
API Data route was acessed
Contact route was acessed
Home route was acessed
```




Pr6
index.js
localhost:3000/about
http://localhost:3000/about
This is the about page


Pr6
index.js
localhost:3000/contact
http://localhost:3000/contact
Contact us at: example@gmail.com

d. Send a JSON response from the server.

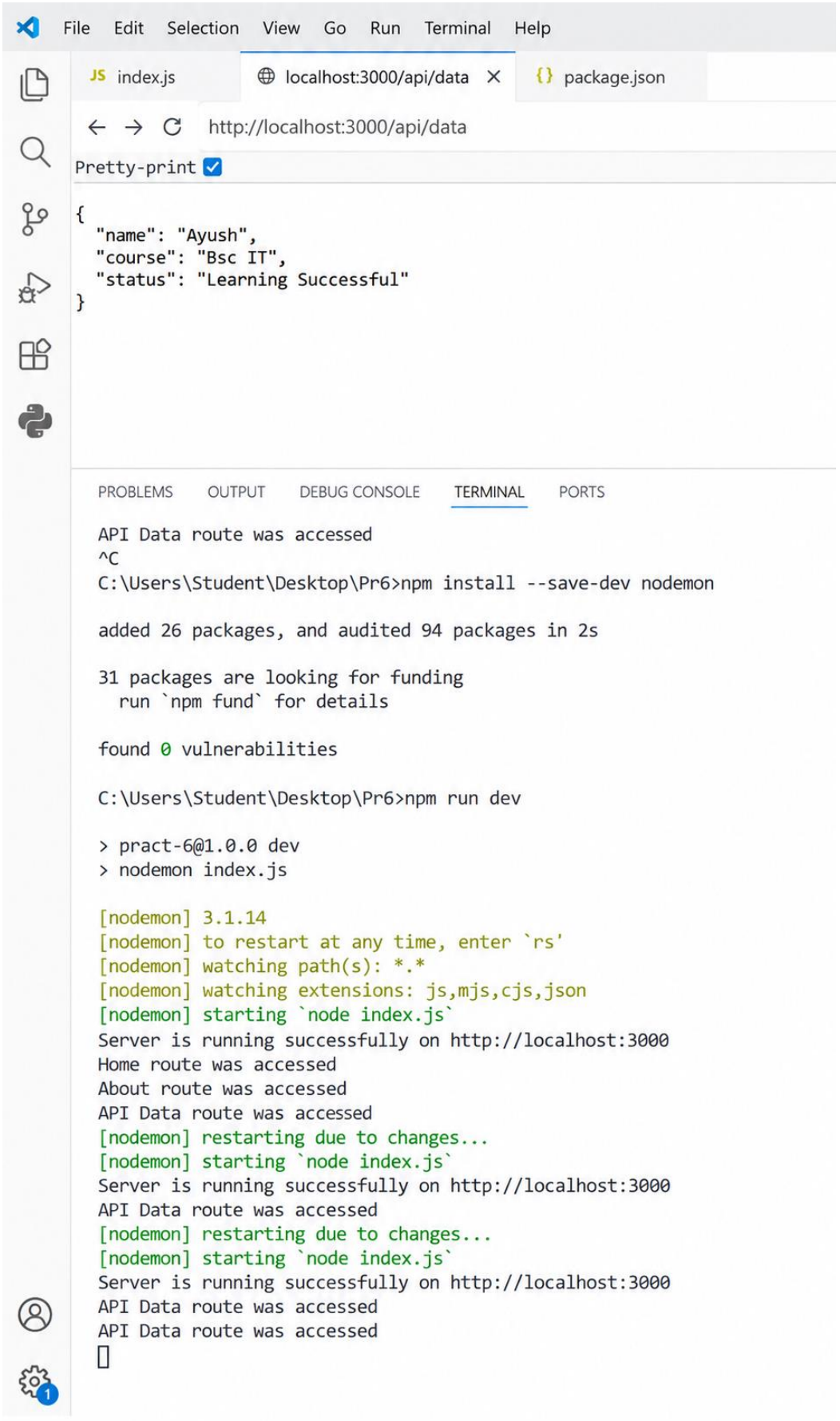


e. Test routes using browser or Postman.
f. Run the server and observe the output in the Console.

Changes in Package.JSON

```
{
  "name": "pract-6",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
  "express": "^5.2.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

Output:


File Edit Selection View Go Run Terminal Help
index.js
localhost:3000/api/data
package.json
http://localhost:3000/api/data
Pretty-print
{
"name": "Ayush",
"course": "Bsc IT",
"status": "Learning Successful"
}
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
API Data route was accessed
^C
C:\Users\Student\Desktop\Pr6>npm install --save-dev nodemon
added 26 packages, and audited 94 packages in 2s
31 packages are looking for funding
run `npm fund` for details
found 0 vulnerabilities
C:\Users\Student\Desktop\Pr6>npm run dev
> pract-6@1.0.0 dev
> nodemon index.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs'
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
Server is running successfully on http://localhost:3000
Home route was accessed
About route was accessed
API Data route was accessed
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Server is running successfully on http://localhost:3000
API Data route was accessed
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Server is running successfully on http://localhost:3000
API Data route was accessed
API Data route was accessed

Practical 7

a. Install and start MongoDB.
b. Create a database and collection.
c. Insert records in MongoDB.
d. View records from the collection.
e. Update and delete records.
f. Connect MongoDB with Node.js.

Code:

```
const { MongoClient } = require('mongodb');

// Connection URL and Database Name
const url = 'mongodb://localhost:27017/';
const dbName = 'Practical7';

async function main() {
  const client = new MongoClient(url);

  try {
    await client.connect();
    console.log('Connected successfully to MongoDB');

    const db = client.db(dbName);
    const collection = db.collection('students');

    // c. Insert a record
    const insertResult = await collection.insertOne({
      name: 'Ayush',
      course: 'BSc IT',
      marks: 92
    });
    console.log('Inserted record:', insertResult.insertedId);

    // d. View records
    const allStudents = await collection.find({}).toArray();
    console.log('All students:', allStudents);

    // e. Update a record
    const updateResult = await collection.updateOne(
      { name: 'Ayush' },
      { $set: { marks: 90 } }
    );
    console.log('Updated count:', updateResult.modifiedCount);

    // f. Delete a record
    const deleteResult = await collection.deleteOne({ name: 'Ayush'});
    console.log('Deleted count:', deleteResult.deletedCount);
```

```
    // View final data
    const finalStudents = await collection.find({}).toArray();
    console.log('Final students list:', finalStudents);

  } catch (err) {
    console.error('Error:', err);
  } finally {
    await client.close();
    console.log('Connection closed');
  }
}

main();
```

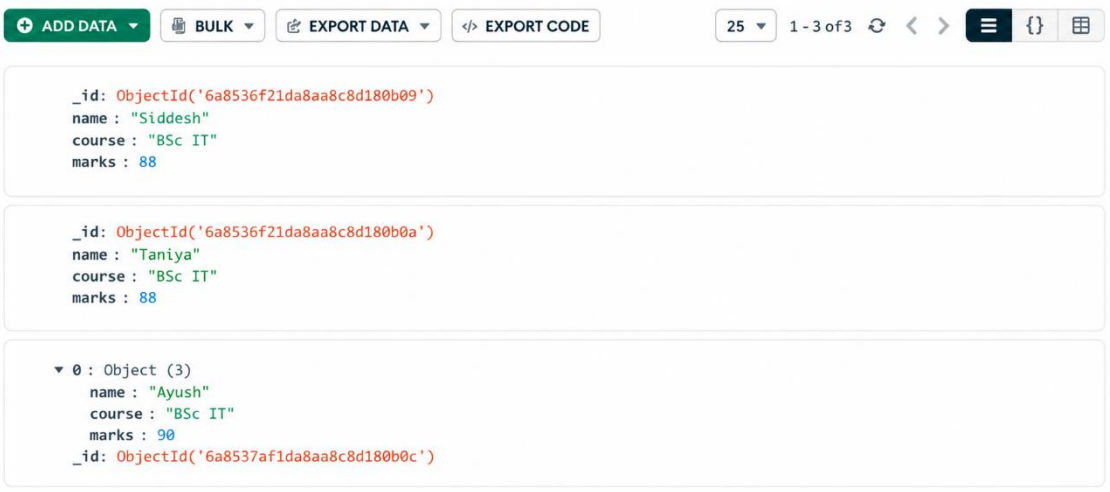
Output:

```
C:\Users\Student\Desktop\Practical7>npm install mongodb

added 12 packages in 3s
```

```
C:\Users\Student\Desktop\Practical7>node Mongo.js
Connected successfully to MongoDB
Inserted record: new ObjectId('6a8535f3c1de061fe36ea255')
All students: [
  {
    _id: new ObjectId('6a8530f787b7f5bf6b9a40d1'),
    name: 'Priya',
    course: 'MongoDB with Node',
    marks: 99
  },
  {
    _id: new ObjectId('6a8535f3c1de061fe36ea255'),
    name: 'Ayush',
    course: 'BSc IT',
    marks: 92
  }
]
Updated count: 1
Deleted count: 1
Final students list: [
  {
    _id: new ObjectId('6a8530f787b7f5bf6b9a40d1'),
    name: 'Priya',
    course: 'MongoDB with Node',
    marks: 99
  }
]
Connection closed

C:\Users\Student\Desktop\Practical7>
```



Practical 8

API Creation and Integration

a. Create a simple REST API using Express
b. Implement a GET API to retrieve data.
c. Fetch data from the API in a React application.
d. Send data from React to the server using a POST request.
e. Store data in MongoDB.
f. Display stored data in React page.

Backend

Server.js

Code:

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const app = express();
const PORT = 5000;
// Middleware
app.use(cors());
app.use(express.json());
// a. Connect to MongoDB
mongoose.connect('mongodb://127.0.0.1:27017/practDB')
  .then(() => console.log('MongoDB connected successfully'))
  .catch((err) => console.error('MongoDB connection error:', err));
// Define Schema and Model
const studentSchema = new mongoose.Schema({
  name: String,
  course: String,
  marks: Number
});

const Student = mongoose.model('Student', studentSchema);
// b. GET API - retrieve all students
app.get('/api/students', async (req, res) => {
try {
    const students = await Student.find();
    res.json(students);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
// d. POST API - add new student
app.post('/api/students', async (req, res) => {
try {
    const newStudent = new Student(req.body);
    const savedStudent = await newStudent.save();
    console.log('New student added:', savedStudent);
```

```
    res.json(savedStudent);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

App.js

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import './App.css';


function App() {
  const [students, setStudents] = useState([]);
  const [name, setName] = useState('');
  const [course, setCourse] = useState('');
  const [marks, setMarks] = useState('');
  // c. Fetch data from API when page loads
  const fetchStudents = async () => {
    try {
      const response = await axios.get('http://localhost:5000/api/students');
      setStudents(response.data);
    } catch (error) {
      console.error('Error fetching students:', error);
    }
  };
  useEffect(() => {
    fetchStudents();
  }, []);
  // d. Send data to server using POST
  const handleSubmit = async (e) => {
  e.preventDefault();
    try {
      await axios.post('http://localhost:5000/api/students', {
        name,
        course,
        marks: Number(marks)
      });
      // Clear form
      setName('');
      setCourse('');
      setMarks('');
      // f. Refresh displayed list after adding
      fetchStudents();
    } catch (error) {
      console.error('Error adding student:', error);
    }
```

```
  };
  return (
    <div style={{ padding: '30px', fontFamily: 'Arial' }}>
      <h1>Student Management</h1>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Name"
          value={name}
          onChange={(e) => setName(e.target.value)}
          required
        />
        <input
          type="text"
          placeholder="Course"
          value={course}
          onChange={(e) => setCourse(e.target.value)}
          required
        />
        <input
          type="number"
          placeholder="Marks"
          value={marks}
          onChange={(e) => setMarks(e.target.value)}
          required
        />
        <button type="submit">Add Student</button>
      </form>
      <h2>Student List</h2>
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>Name</th>
            <th>Course</th>
            <th>Marks</th>
          </tr>
        </thead>
        <tbody>
          {students.map((student) => (
            <tr key={student._id}>
              <td>{student.name}</td>
              <td>{student.course}</td>
              <td>{student.marks}</td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

export default App;

Output:

```
Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Asus\OneDrive\Desktop\MERN 8>mkdir backend

C:\Users\Asus\OneDrive\Desktop\MERN 8>mkdir frontend

C:\Users\Asus\OneDrive\Desktop\MERN 8>cd backend

C:\Users\Asus\OneDrive\Desktop\MERN 8\backend>npm init -y
Wrote to C:\Users\Asus\OneDrive\Desktop\MERN 8\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}


C:\Users\Asus\OneDrive\Desktop\MERN 8\backend>npm install express mongoose cors

added 88 packages, and audited 89 packages in 10s

29 packages are looking for funding
  run `npm fund` for details
```

Problems Output Debug Console Terminal Ports

```
C:\Users\Asus\OneDrive\Desktop\MERN 8\backend>npm install express mongoose cors

added 88 packages, and audited 89 packages in 10s

29 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Asus\OneDrive\Desktop\MERN 8\backend>node server.js
Server running on http://localhost:5000
MongoDB connected successfully
New student added: {
  name: 'Ayush',
  course: 'Bsc IT',
  marks: 99,
  _id: new ObjectId('6a9c16d42732042172fcdc33'),
  __v: 0
}
```

Problems Output Debug Console Terminal Ports

```
Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Asus\OneDrive\Desktop\MERN 8>cd frontend

C:\Users\Asus\OneDrive\Desktop\MERN 8\frontend>npm create vite@latest . -- --template react

> npx
> create-vite . --template react

◇ Which linter to use?
  ESLint

◇ Install with npm and start now?
  Yes

◇ Scaffolding project in C:\Users\Asus\OneDrive\Desktop\MERN 8\frontend...

◇ Installing dependencies with npm...

added 143 packages, and audited 144 packages in 23s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

◇ Starting dev server...

> frontend@0.0.0 dev
> vite

  VITE v8.2.2  ready in 797 ms
```

```
  ➜ Local:   http://localhost:5173/
  ➜ Network: use --host to expose
  ➜ press h + enter to show help
Terminate batch job (Y/N)? y

C:\Users\Asus\OneDrive\Desktop\MERN 8\frontend>npm install

up to date, audited 144 packages in 1s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Asus\OneDrive\Desktop\MERN 8\frontend>npm install axios

added 25 packages, and audited 169 packages in 3s

37 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

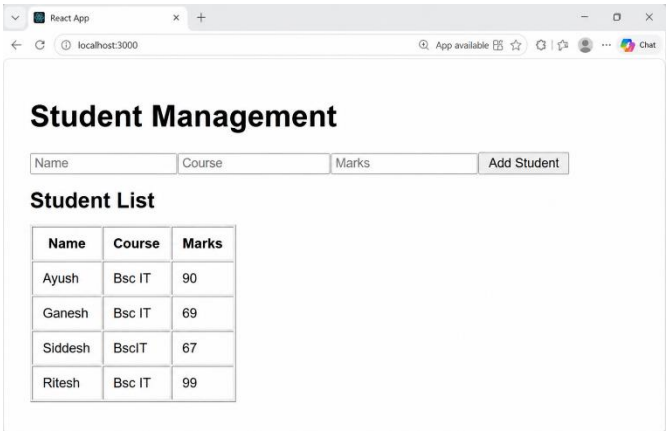
C:\Users\Asus\OneDrive\Desktop\MERN 8\frontend>npm run dev

> frontend@0.0.0 dev
> vite

6:48:57 pm [vite] (client) Re-optimizing dependencies because lockfile has changed

  VITE v8.2.2  ready in 1193 ms

  ➜ Local:   http://localhost:5173/
```



Practical 9

User Data Management

a. Create an API to add user details using Express and MongoDB.
b. Design a React form to send user data to the backend.
c. Retrieve and display user data from MongoDB.
d. Implement an update operation for existing records.
e. Implement the delete operation for selected records.
f. Display updated records dynamically in React.

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

// MongoDB connection
mongoose
  .connect("mongodb://127.0.0.1:27017/user_management")
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));

// User Schema
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
  },
  email: {
    type: String,
    required: true,
  },
```

```
  age: {
    type: Number,
    required: true,
  },
});

const User = mongoose.model("User", userSchema);

// CREATE user
app.post("/users", async (req, res) => {
  try {
    const user = new User(req.body);
    await user.save();

    res.status(201).json(user);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// READ users
app.get("/users", async (req, res) => {
  try {
    const users = await User.find();
    res.json(users);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

// UPDATE user
app.put("/users/:id", async (req, res) => {
  try {
    const updatedUser = await User.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true }
    );

    res.json(updatedUser);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
```

```
});

// DELETE user
app.delete("/users/:id", async (req, res) => {
  try {
    await User.findByIdAndDelete(req.params.id);

    res.json({ message: "User deleted successfully" });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
});

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

App.jsx

```
import { useEffect, useState } from "react";
import "./App.css";

function App() {
  const [users, setUsers] = useState([]);

  const [formData, setFormData] = useState({
    name: "",
    email: "",
    age: "",
  });

  const [editingId, setEditingId] = useState(null);

  // Get users
  const fetchUsers = async () => {
    const response = await fetch("http://localhost:5000/users");
    const data = await response.json();

    setUsers(data);
  };

  useEffect(() => {
    fetchUsers();
  }, []);
```

```
  // Handle input
  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value,
    });
  };

  // Add / Update user
  const handleSubmit = async (e) => {
    e.preventDefault();

    const url = editingId
      ? `http://localhost:5000/users/${editingId}`
      : "http://localhost:5000/users";

    const method = editingId ? "PUT" : "POST";

    const response = await fetch(url, {
      method: method,
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        ...formData,
        age: Number(formData.age),
      }),
    });

    if (response.ok) {
      setFormData({
      name: "",
        email: "",
        age: "",
      });

      setEditingId(null);

      fetchUsers();
    }
  };
```

```
  // Edit user
  const handleEdit = (user) => {
    setFormData({
      name: user.name,
      email: user.email,
      age: user.age,
    });

    setEditingId(user._id);
  };

  // Delete user
  const handleDelete = async (id) => {
    await fetch(`http://localhost:5000/users/${id}`, {
      method: "DELETE",
    });

    fetchUsers();
  };

  return (
    <div className="container">
      <h1>User Management</h1>

      <form onSubmit={handleSubmit}>
        <input
          type="text"
          name="name"
          placeholder="Enter name"
          value={formData.name}
          onChange={handleChange}
          required
        />

        <input
          type="email"
          name="email"
          placeholder="Enter email"
          value={formData.email}
          onChange={handleChange}
          required
        />
```

```
        <input
          type="number"
          name="age"
          placeholder="Enter age"
          value={formData.age}
          onChange={handleChange}
          required
        />

        <button type="submit">
          {editingId ? "Update User" : "Add User"}
        </button>
      </form>

      <h2>Users</h2>

      <table>
        <thead>
          <tr>
            <th>Name</th>
            <th>Email</th>
            <th>Age</th>
            <th>Actions</th>
          </tr>
        </thead>

        <tbody>
          {users.map((user) => (
            <tr key={user._id}>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>{user.age}</td>

              <td>
                <button onClick={() => handleEdit(user)}>
                  Edit
                </button>

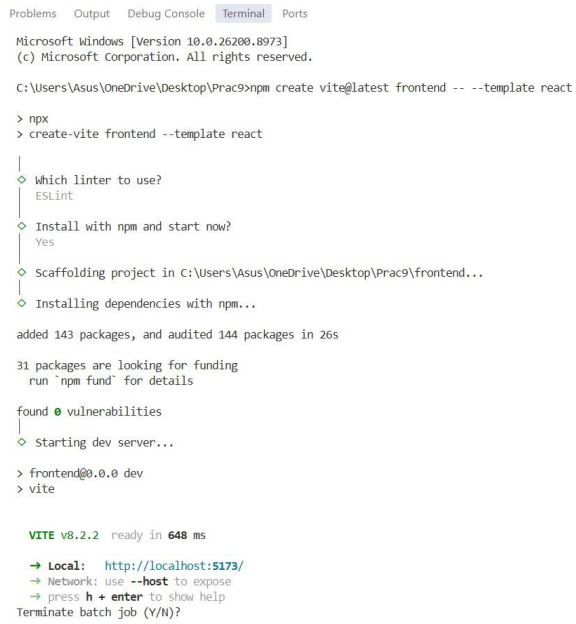
                <button onClick={() => handleDelete(user._id)}>
                  Delete
                </button>
              </td>
            </tr>
```

```
        ))}
      </tbody>
    </table>
  </div>

);
}

export default App;
```





```
C:\Users\Asus\OneDrive\Desktop\Prac9>cd frontend

C:\Users\Asus\OneDrive\Desktop\Prac9\frontend>npm install

up to date, audited 144 packages in 2s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Asus\OneDrive\Desktop\Prac9\frontend>cd ..

C:\Users\Asus\OneDrive\Desktop\Prac9>mkdir backend

C:\Users\Asus\OneDrive\Desktop\Prac9>cd backend

C:\Users\Asus\OneDrive\Desktop\Prac9\backend>npm init -y
Wrote to C:\Users\Asus\OneDrive\Desktop\Prac9\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
C:\Users\Asus\OneDrive\Desktop\Prac9\backend>npm install express mongoose cors dotenv

added 89 packages, and audited 90 packages in 11s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Asus\OneDrive\Desktop\Prac9\backend>npm install --save-dev nodemon

added 26 packages, and audited 116 packages in 4s

35 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Asus\OneDrive\Desktop\Prac9\backend>node server.js
Server running on port 5000
MongoDB Connected
```

App.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
}

.container {
```

```
  width: 800px;
  margin: 50px auto;
  background: white;
  padding: 30px;
  border-radius: 10px;
}

h1,
h2 {
  text-align: center;
}

form {
  display: flex;
  gap: 10px;
  margin-bottom: 30px;
}

input {
  padding: 10px;
  flex: 1;
}

button {
  padding: 10px 15px;
  border: none;
  background-color: #007bff;
  color: white;
  cursor: pointer;
  border-radius: 5px;
}

button:hover {
  background-color: #0056b3;
}

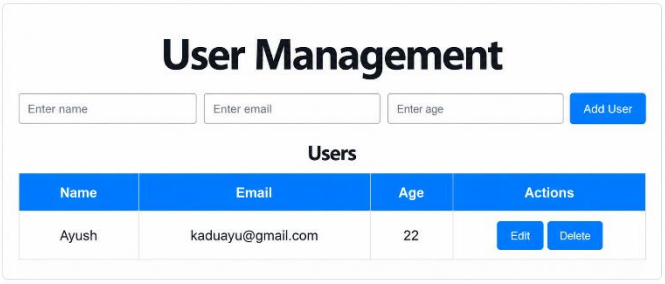
table {
  width: 100%;
  border-collapse: collapse;
}

th,
td {
```

```
  padding: 12px;
  border: 1px solid #ddd;
  text-align: center;
}

th {
  background-color: #007bff;
  color: white;
}

td button {
  margin: 3px;
```



Practical 10

a. Integrate React frontend with Node and Express backend.
b. Perform insert, update, and delete operations through UI.
c. Display database records dynamically in React.
d. Implement frontend form validation.
e. Test all APIs using Postman.
f. Execute and verify the complete working of the frontend, backend, and database together.

Code:

Student.js

```
const mongoose = require("mongoose");

const studentSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true
    },

    email: {
      type: String,
      required: true,
      unique: true,
      trim: true
    },

    age: {
      type: Number,
      required: true,
      min: 1
    }
  },
  {
    timestamps: true
  }
```

```
);

module.exports = mongoose.model("Student", studentSchema);
```

studentsroutes.js

```
const express = require("express");
const Student = require("../models/Student");

const router = express.Router();

// GET all students
router.get("/", async (req, res) => {
  try {
    const students = await Student.find();
    res.json(students);
  } catch (error) {
    res.status(500).json({
    message: error.message
    });
  }
});

// GET student by ID
router.get("/:id", async (req, res) => {
try {
    const student = await Student.findById(req.params.id);

    if (!student) {
      return res.status(404).json({
        message: "Student not found"
      });
    }

    res.json(student);
  } catch (error) {
    res.status(500).json({
    message: error.message
    });
  }
});

// INSERT student
router.post("/", async (req, res) => {
```

```
  try {
    const { name, email, age } = req.body;

    const student = new Student({
      name,
      email,
      age
    });

    const savedStudent = await student.save();

    res.status(201).json(savedStudent);
  } catch (error) {
    res.status(400).json({
    message: error.message
    });
  }
});

// UPDATE student
router.put("/:id", async (req, res) => {
  try {
    const updatedStudent = await Student.findByIdAndUpdate(
      req.params.id,
      req.body,
      {
        new: true,
        runValidators: true
      }
    );

    if (!updatedStudent) {
      return res.status(404).json({
        message: "Student not found"
      });
    }

    res.json(updatedStudent);
  } catch (error) {
    res.status(400).json({
    message: error.message
    });
  }
```

```
});

// DELETE student
router.delete("/:id", async (req, res) => {
  try {
    const deletedStudent = await Student.findByIdAndDelete(
      req.params.id
    );

    if (!deletedStudent) {
      return res.status(404).json({
        message: "Student not found"
      });
    }

    res.json({
      message: "Student deleted successfully"
    });
  } catch (error) {
    res.status(500).json({
    message: error.message
    });
  }
});

module.exports = router;
```

server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const dotenv = require("dotenv");

const studentRoutes = require("./routes/studentRoutes");

dotenv.config();

const app = express();

// Middleware
app.use(cors());
app.use(express.json());
```

```
// Routes
app.use("/api/students", studentRoutes);

// Test route
app.get("/", (req, res) => {
  res.send("Backend server is running");
});

// MongoDB connection
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    console.log("MongoDB connected");

    app.listen(process.env.PORT, () => {
      console.log(
        `Server running on http://localhost:${process.env.PORT}`
      );
    });
  })
  .catch((error) => {
    console.error("MongoDB connection failed:", error);
  });
```

.env

```
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/studentdb
```

App.jsx

```
import { useEffect, useState } from "react";
import "./App.css";

const API_URL = "http://localhost:5000/api/students";

function App() {
  const [students, setStudents] = useState([]);

  const [form, setForm] = useState({
    name: "",
    email: "",
    age: ""
  });
```

```
  const [editingId, setEditingId] = useState(null);
  const [error, setError] = useState("");

  // Get students
  const fetchStudents = async () => {
    try {
      const response = await fetch(API_URL);
      const data = await response.json();

      setStudents(data);
    } catch (error) {
      setError("Unable to fetch students");
    }
  };

  // Load students when page opens
  useEffect(() => {
    fetchStudents();
  }, []);

  // Handle input
  const handleChange = (e) => {
    setForm({
      ...form,
      [e.target.name]: e.target.value
    });
  };

  // Frontend validation
  const validateForm = () => {
    if (!form.name.trim()) {
      setError("Name is required");
      return false;
    }

    if (!form.email.trim()) {
      setError("Email is required");
      return false;
    }

    const emailPattern =
      /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
```

```
    if (!emailPattern.test(form.email)) {
      setError("Enter a valid email address");
      return false;
    }

    if (!form.age) {
      setError("Age is required");
      return false;
    }

    if (Number(form.age) < 1) {
      setError("Age must be greater than 0");
      return false;
    }

    setError("");
    return true;
  };

  // Insert / Update
  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!validateForm()) {
      return;
    }

    try {
      const url = editingId
        ? `${API_URL}/${editingId}`
        : API_URL;

      const method = editingId ? "PUT" : "POST";

      const response = await fetch(url, {
        method,
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          name: form.name,
          email: form.email,
          age: Number(form.age)
```

```
      })
    });

    const data = await response.json();

    if (!response.ok) {
      setError(data.message || "Operation failed");
      return;
    }

    // Clear form
    setForm({
    name: "",
      email: "",
      age: ""
    });

    setEditingId(null);
    setError("");

    // Refresh database records
    fetchStudents();
  } catch (error) {
    setError("Server error");
  }
};

// Edit
const handleEdit = (student) => {
  setForm({
    name:  student.name,
    email: student.email,
    age: student.age
  });

  setEditingId(student._id);
};

// Delete
const handleDelete = async (id) => {
  const confirmDelete = window.confirm(
    "Are you sure you want to delete this student?"
  );
```

```
    if (!confirmDelete) {
      return;
    }

    try {
      const response = await fetch(`${API_URL}/${id}`, {
        method: "DELETE"
      });

      const data = await response.json();

      if (!response.ok) {
        setError(data.message || "Delete failed");
        return;
      }

      fetchStudents();
    } catch (error) {
      setError("Server error");
    }
  };

  return (
    <div className="container">
      <h1>Student Management System</h1>

      <form onSubmit={handleSubmit}>
        <input
          type="text"
          name="name"
          placeholder="Enter name"
          value={form.name}
          onChange={handleChange}
        />

        <input
          type="email"
          name="email"
          placeholder="Enter email"
          value={form.email}
          onChange={handleChange}
        />
```

```
      <input
        type="number"
        name="age"
        placeholder="Enter age"
        value={form.age}
        onChange={handleChange}
      />

      <button type="submit">
        {editingId ? "Update Student" : "Add Student"}
      </button>
    </form>

    {error && (
      <p className="error">
        {error}
      </p>
    )}

    <h2>Student Records</h2>

    <table>
      <thead>
        <tr>
          <th>Name</th>
          <th>Email</th>
          <th>Age</th>
          <th>Actions</th>
        </tr>
      </thead>

      <tbody>
        {students.map((student) => (
          <tr key={student._id}>
            <td>{student.name}</td>
            <td>{student.email}</td>
            <td>{student.age}</td>

            <td>
              <button
                onClick={() => handleEdit(student)}
              >
```

```
              Edit
            </button>

            <button
              onClick={() =>
                handleDelete(student._id)
              }
            >
              Delete
            </button>
          </td>
        </tr>
      ))}
    </tbody>
  </table>
 </div>
 );
}

export default App;
```

App.css

```
body {
  font-family: Arial, sans-serif;
  background: #f4f4f4;
}

.container {
  width: 800px;
  margin: 40px auto;
  background: white;
  padding: 30px;
  border-radius: 10px;
}

h1,
h2 {
  text-align: center;
}

form {
  display: flex;
  gap: 10px;
```

```
  margin-bottom: 20px;
}

input {
  padding: 10px;
  flex: 1;
}

button {
  padding: 10px 15px;
  border: none;
  background: #007bff;
  color: white;
  cursor: pointer;
  border-radius: 5px;
}

button:hover {
  background: #0056b3;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th,
td {
  padding: 12px;
  border: 1px solid #ddd;
  text-align: center;
}

.error {
  color: red;
  text-align: center;
}
```

```
C:\Users\Asus\OneDrive\Desktop\MERN10>mkdir fullstack-project

C:\Users\Asus\OneDrive\Desktop\MERN10>cd fullstack-project

C:\Users\Asus\OneDrive\Desktop\MERN10\fullstack-project>mkdir backend

C:\Users\Asus\OneDrive\Desktop\MERN10\fullstack-project>cd backend

C:\Users\Asus\OneDrive\Desktop\MERN10\fullstack-project\backend>npm init -y
Wrote to C:\Users\Asus\OneDrive\Desktop\MERN10\fullstack-project\backend\package.json:

{
  "name": "backend",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}


C:\Users\Asus\OneDrive\Desktop\MERN10\fullstack-project\backend>npm install express mongoose cors dotenv

added 89 packages, and audited 90 packages in 10s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Problems Output Debug Console Terminal Ports

```
Microsoft Windows [Version 10.0.26200.8973]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Asus\OneDrive\Desktop\MERN10>cd fullstack-project

C:\Users\Asus\OneDrive\Desktop\MERN10\fullstack-project>npm create vite@latest frontend -- --template react

> npx
> create-vite frontend --template react

│
◇  Which linter to use?
│  ESLint
│
◇  Install with npm and start now?
│  Yes
│
◇  Scaffolding project in C:\Users\Asus\OneDrive\Desktop\MERN10\fullstack-project\frontend...
│
◇  Installing dependencies with npm...

added 143 packages, and audited 144 packages in 24s

31 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
│
◇  Starting dev server...

> frontend@0.0.0 dev
> vite


  VITE v8.2.2  ready in 1243 ms

  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

```
C:\Users\Asus\OneDrive\Desktop\MERN10\fullstack-project\backend>npm install --save-dev nodemon

added 26 packages, and audited 116 packages in 4s

35 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
> dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
◇ injected env (2) from .env // tip: ⌁ auth for agents [www.vestauth.com]
MongoDB connected
Server running on http://localhost:5000
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (2) from .env // tip: ⌘ multiple files { path: ['.env.local', '.env'] }
MongoDB connected
Server running on http://localhost:5000
```

